Practice Manual 1

Build a Static Website with Django

Introduction to Python | Django Web Framework

You will build	A two-page personal website (Home and About) with colours and a navigation bar
You need	A laptop with Python 3.12 or newer, VS Code, and a web browser
Time	About 90 minutes
Level	Complete beginner. No web development experience is needed
Names used	Folder: `practice1_website` Project: `config` App: `pages`

What Is Django?

Django is a **web framework** written in Python. A framework is a ready-made toolkit. Instead of building every part of a website from scratch, you use Django's tools and write only the parts that make your website unique.

Every website works like a conversation. Your browser sends a **request** ("please show me the About page") and a server sends back a **response** (the page). Django is the program on the server that receives the request and decides what to send back.

In this manual you will use three building blocks:

Building block	Its job	Where it lives
URL	Reads the address typed in the browser and decides which view should handle it	`urls.py`
View	A Python function that receives the request and returns a response	`views.py`
Template	An HTML file that decides how the page looks	`templates` folder

How to Use This Manual

Follow the steps in order and do not skip any. Each step tells you **what to do** and, in the blue box, **why you are doing it**. The coloured boxes mean:

> **WHY WE DO THIS** Explains the reason behind a step. Read it: understanding the reason is how you learn.

> **CHECKPOINT** Tells you what you should see before moving to the next step. If you do not see it, stop and fix it first.

> **TIP** A helpful shortcut or extra information.

WATCH OUT A common mistake. Read this before you continue.

Three rules that prevent most problems:

1. **Type carefully.** Python is sensitive to spaces at the start of a line (indentation), to quotes, and to commas.
2. **Save every file.** Turn on Auto Save in Step 1. Django can only use code that has been saved.
3. **Read the last line of any error message first.** It usually names the problem. Common errors are listed at the end of this manual.

Part A: Setting Up Your Workspace

Before writing any website code, we prepare a clean workspace. These seven steps are done once. Do not skip them, because most beginner errors come from a missed setup step.

Step 1: Create a Folder and Open It in VS Code

1. On your Desktop, create a new folder named `practice1_website`.
2. Open VS Code. Click **File** → **Open Folder**, select `practice1_website`, and click **Open**.
3. Open the terminal inside VS Code: click **Terminal** → **New Terminal**. A panel opens at the bottom. This is where you will type commands.
4. Turn on Auto Save: click **File** → **Auto Save** and make sure a tick appears beside it.

> **WHY WE DO THIS** All files for this website live in one folder, so everything stays together. The terminal lets you give commands to your computer without leaving VS Code. Auto Save matters because Django only sees what has been saved. Unsaved code is the most common reason for "my code does not work".

> **TIP** A small white dot on a file's tab means the file has unsaved changes. Press Cmd + S (Mac) or Ctrl + S (Windows) to save.

Step 2: Create a Virtual Environment

Type these commands in the terminal, one at a time, pressing Enter after each. Use the column that matches your computer.

Task	Mac	Windows
Create it	`python3 -m venv venv`	`python -m venv venv`
Activate it	`source venv/bin/activate`	`venv\Scripts\activate`

> **WHY WE DO THIS** A virtual environment is a private toolbox for one project. Django and other packages are installed inside it, not on your whole computer, so different projects never interfere with each other.

> **CHECKPOINT** The terminal line now starts with `(venv)`, and a new folder named `venv` appears in the Explorer panel on the left.

> **WATCH OUT** Every time you close VS Code and come back, you must activate the environment again. If `(venv)` is missing, run the Activate command again. On Windows PowerShell, if activation is blocked, run `Set-ExecutionPolicy -Scope Process -ExecutionPolicy Bypass` and then activate again.

Step 3: Install Django

TERMINAL **Type in the terminal**

```
pip install django
python -m django --version
```

The first command downloads and installs Django. The second prints the installed version number, which proves the installation worked. From now on, while `(venv)` is showing, use the word `python` on both Mac and Windows.

> **CHECKPOINT** A version number is printed (for example 5.2 or 6.1). The exact number does not matter.

Step 4: Tell VS Code to Use the Virtual Environment

1. Press **Cmd + Shift + P** (Mac) or **Ctrl + Shift + P** (Windows).
2. Type `Python: Select Interpreter` and press Enter.
3. Choose the option that mentions `venv` (for example `./venv/bin/python`).

> **WHY WE DO THIS** VS Code needs to know where Django is installed. Without this, it may underline Django code in red even when the code is correct.

> **TIP** If you do not see the Python option, install the extension: click the Extensions icon on the left, search for **Python** (by Microsoft), and click Install.

Step 5: Create the Django Project

TERMINAL **Type in the terminal**

```
django-admin startproject config .
```

Notice the dot (`.`) at the end. It means “create the project right here in this folder”. Without the dot, Django creates an extra folder inside, and the paths in this manual will not match.

File	What it is
`manage.py`	The control panel. You use it to run commands such as starting the server.
`config/settings.py`	All project settings, such as which apps are installed.
`config/urls.py`	The main address book of the website.

> **WHY WE DO THIS** A Django project is the whole website. The `startproject` command builds the standard skeleton so you do not have to create these files by hand.

> **CHECKPOINT** The Explorer panel now shows a `config` folder and a `manage.py` file.

Step 6: Create the App

TERMINAL **Type in the terminal**

```
python manage.py startapp pages
```

> **WHY WE DO THIS** A Django project is made of small parts called **apps**, and each app handles one feature. Our app is named `pages` because it holds the pages of our website. Think of the project as a college building and each app as one department inside it.

CHECKPOINT A `pages` folder appears containing files such as `views.py` and `models.py`. Today we use `views.py`. The file `models.py` is used in Practice 2.

Step 7: Register the App

Open `config/settings.py`. Press Cmd/Ctrl + F, search for `INSTALLED_APPS`, and add the line `'pages',` at the end of the list, exactly as shown:

FILE **config/settings.py**

```
INSTALLED_APPS = [
    'django.contrib.admin',
    'django.contrib.auth',
    'django.contrib.contenttypes',
    'django.contrib.sessions',
    'django.contrib.messages',
    'django.contrib.staticfiles',
    'pages',
]
```

WHY WE DO THIS Creating an app is not enough. Django only uses the apps listed in `INSTALLED_APPS`, so this line introduces our app to the project.

WATCH OUT Keep the quotes and the comma. A missing comma is a very common mistake.

Part B: Your First Page

We now make the smallest possible web page: one that only shows a line of text. It teaches the most important idea in Django, which is how a request travels from the browser to your Python code.

Step 8: Write a View

Open `pages/views.py`, delete everything in it, and type:

FILE **pages/views.py**

```
from django.http import HttpResponse

def home(request):
    return HttpResponse("Hello! This is my first Django page.")
```

- `from django.http import HttpResponse` brings in Django's tool for sending text back to the browser.
- `def home(request):` creates a Python function named `home`. Django hands it the browser's request automatically.
- `return HttpResponse(...)` sends the text back to the browser as the reply.

WHY WE DO THIS A **view** is simply a Python function that answers a request. Whatever the function returns is what the visitor sees.

Step 9: Create the App's URL File

In the Explorer, right-click the `pages` folder, choose **New File**, name it `urls.py`, and type:

FILE `pages/urls.py`

```
from django.urls import path
from . import views

urlpatterns = [
    path('', views.home, name='home'),
]
```

- `path('', ...)` the empty quotes mean "the main page of the site".
- `views.home` is the function we wrote in Step 8.
- `name='home'` is a nickname for this address. We will use it later to make links.

WHY WE DO THIS A view on its own cannot be reached. A URL pattern connects a web address to a view.

WATCH OUT The list must be named exactly `urlpatterns` (all lowercase, ending with s). Django looks for this exact name.

Step 10: Connect It to the Main URL File

Open `config/urls.py`, delete everything, and type:

FILE `config/urls.py`

```
from django.contrib import admin
from django.urls import path, include

urlpatterns = [
    path('admin/', admin.site.urls),
    path('', include('pages.urls')),
]
```

WHY WE DO THIS Django checks `config/urls.py` first for every request. The line `include('pages.urls')` says: "for everything else, look inside the pages app's own URL file". Each app keeps its own addresses in its own file, which stays tidy as the site grows.

Step 11: Run the Server

TERMINAL **Type in the terminal**

```
python manage.py runserver
```

Now open your browser and go to:

BROWSER **Type in the address bar**

```
http://127.0.0.1:8000/
```

CHECKPOINT The browser shows: **Hello! This is my first Django page.**

WHY WE DO THIS `runserver` starts a small web server on your own laptop, so you can see your website without internet. `127.0.0.1` always means "this computer" and `8000` is the port number (the door the server listens on).

> **TIP** Red text about “unapplied migration(s)” is only a warning, not an error. It is about the database, which this website does not use. You will handle it in Practice 2.

> **TIP** Leave the server running. It restarts automatically every time you save a file. To stop it, click in the terminal and press **Ctrl + C**. To start it again, run the same command.

What just happened? Follow the journey of your request:

1. You typed the address. The browser sent a request to your server.
2. `config/urls.py` found `include('pages.urls')` and passed the request on.
3. `pages/urls.py` matched the empty address and picked `views.home`.
4. `home()` ran and returned the text, which the browser displayed.

Part C: From Text to a Real Web Page

Plain text is not much of a website. Real pages are written in HTML. In Django, HTML files live in **templates**. We will move the page design into a template and let the view display it.

Step 12: Create the Templates Folders and home.html

1. Right-click the `pages` folder, choose **New Folder**, and name it `templates`.
2. Right-click the new `templates` folder, choose **New Folder**, and name it `pages`.
3. Right-click that inner `pages` folder, choose **New File**, and name it `home.html`.

The final path must be `pages/templates/pages/home.html`. Type this in the file:

FILE **pages/templates/pages/home.html**

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <title>My Website</title>
</head>
<body>
    <h1>Welcome to My Website</h1>
    <p>Hello! My name is Your Name. I am learning Django.</p>

    <h2>My Skills</h2>
    <ul>
        <li>Python</li>
        <li>HTML</li>
        <li>Django</li>
    </ul>
</body>
</html>
```

> **WHY WE DO THIS** A template is an HTML file that describes how a page looks. Django automatically looks inside the `templates` folder of every app. The extra inner `pages` folder is a Django habit: it gives each template a unique name (`pages/home.html`), so two apps that both have a `home.html` never get mixed up.

WATCH OUT The folder must be named `templates` (with an s) and must sit inside the `pages` app folder. A wrong name causes the error `TemplateDoesNotExist`.

Step 13: Make the View Show the Template

Open `pages/views.py`, delete everything, and type:

FILE `pages/views.py`

```
from django.shortcuts import render

def home(request):
    return render(request, 'pages/home.html')
```

WHY WE DO THIS `HttpResponse` could only send plain text. `render` finds the template, prepares it, and sends the finished HTML page to the browser. The value `'pages/home.html'` is the template's path inside the `templates` folder.

CHECKPOINT Refresh the browser (Cmd + R or Ctrl + R). You now see a heading, a paragraph, and a bulleted skills list.

TIP Try it: change "Your Name" in `home.html` to your own name, save, and refresh the browser.

Part D: A Second Page and Navigation

Now we add an About page and links to move between pages. This shows the pattern you will repeat for every new page.

Step 14: Create about.html

In the same folder (`pages/templates/pages/`), create a new file named `about.html` and type:

FILE `pages/templates/pages/about.html`

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <title>About Me</title>
</head>
<body>
    <h1>About Me</h1>
    <p>I study Computer Science and Engineering.</p>
    <p>I am learning Python and building websites with Django.</p>
    <p>My hobbies are reading, cricket, and coding.</p>
</body>
</html>
```

Step 15: Add the About View

Open `pages/views.py`, delete everything, and type both views:

FILE **pages/views.py**

```
from django.shortcuts import render

def home(request):
    return render(request, 'pages/home.html')

def about(request):
    return render(request, 'pages/about.html')
```

Step 16: Add the About URL

Open `pages/urls.py`, delete everything, and type:

FILE **pages/urls.py**

```
from django.urls import path
from . import views

urlpatterns = [
    path('', views.home, name='home'),
    path('about/', views.about, name='about'),
]
```

Now visit the new page in your browser:

BROWSER **Type in the address bar**

```
http://127.0.0.1:8000/about/
```

> **WHY WE DO THIS** **Remember this pattern.** Every new page needs three things: (1) a template that holds the HTML, (2) a view that renders it, and (3) a URL that points to the view. You have just done all three for the About page.

Step 17: Add Navigation Links

Open **both** `home.html` and `about.html`. In each file, add this block directly below the line `<body>`:

FILE **home.html and about.html (add below <body>)**

```
    <nav>
        <a href="{% url 'home' %}">Home</a>
        <a href="{% url 'about' %}">About</a>
    </nav>
```

The top of `home.html` should now look like this:

FILE **home.html (after the change)**

```
<body>
    <nav>
        <a href="{% url 'home' %}">Home</a>
        <a href="{% url 'about' %}">About</a>
    </nav>

    <h1>Welcome to My Website</h1>
```

WHY WE DO THIS `{% url 'home' %}` is Django template code. It looks up the URL whose `name` is `'home'` (the nickname we gave in `pages/urls.py`) and fills in the real address. If you change an address later, the links fix themselves. In templates, `{% ... %}` gives Django an instruction and `{{ ... }}` displays a value.

CHECKPOINT Both pages show Home and About links, and clicking them switches between the pages.

WATCH OUT Add the block to both files. If a name inside `{% url %}` does not match a `name=` in `urls.py`, you get the error `NoReverseMatch`.

Part E: Adding Style with CSS

The pages work but look plain. HTML holds the content; **CSS** holds the design (colours, spacing, fonts). Files that never change, such as CSS, images, and JavaScript, are called **static files**.

Step 18: Create the CSS File

1. Right-click the `pages` folder, choose **New Folder**, and name it `static`.
2. Inside `static`, create a folder named `pages`.
3. Inside that `pages` folder, create a new file named `style.css`.

The final path must be `pages/static/pages/style.css`. Type this in the file:

FILE **pages/static/pages/style.css**

```
/* Whole page */
body {
    font-family: Arial, sans-serif;
    max-width: 700px;
    margin: 40px auto;
    padding: 0 20px;
    background-color: #f5f7fa;
    color: #333333;
}

/* Navigation bar */
nav {
    background-color: #1f4e79;
    padding: 12px 16px;
    border-radius: 6px;
    margin-bottom: 24px;
}

nav a {
    color: #ffffff;
    text-decoration: none;
    margin-right: 16px;
    font-weight: bold;
}

/* Headings and list items */
h1 {
    color: #1f4e79;
}

h2 {
    color: #2e75b6;
}

li {
    margin-bottom: 6px;
}
```

WHY WE DO THIS Django looks for static files in a folder named `static` inside each app. We use the same double-folder habit as with templates, so file names never clash between apps.

Step 19: Connect the CSS to Both Pages

In **both** `home.html` and `about.html`, make two changes: add `{% load static %}` as the very first line, and add the `<link>` line inside `<head>`. The top of each file should look like this:

FILE **home.html and about.html (top of file)**

```
{% load static %}
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <title>My Website</title>
    <link rel="stylesheet" href="{% static 'pages/style.css' %}">
</head>
```

WHY WE DO THIS `{% load static %}` switches on Django's static-file tools for this page. The `<link>` tag asks the browser to download `style.css` and apply it. The `{% static %}` tag builds the correct address of the file.

CHECKPOINT Refresh the browser. You now see a navy navigation bar, coloured headings, and a grey-blue background on both pages.

TIP If the style does not appear, stop the server (Ctrl + C in the terminal), start it again with `python manage.py runserver`, and do a hard refresh: Cmd + Shift + R (Mac) or Ctrl + F5 (Windows).

Part F: Bonus, Send Data from Python to the Page

So far, all text was typed directly into HTML files. Real websites usually get their information from Python code or a database. This bonus part shows how Python passes data into a template. Practice 2 uses exactly the same idea with a database.

Step 20: Send Data from the View

Open `pages/views.py`, delete everything, and type:

FILE **pages/views.py**

```
from django.shortcuts import render

def home(request):
    context = {
        'student_name': 'Your Name Here',
        'skills': ['Python', 'HTML', 'Django'],
    }
    return render(request, 'pages/home.html', context)

def about(request):
    return render(request, 'pages/about.html')
```

Step 21: Show the Data in home.html

Make two replacements in `home.html`. First, replace the paragraph line with:

FILE **pages/templates/pages/home.html (replace the <p> line)**

```
    <p>Hello! My name is {{ student_name }}. I am learning Django.</p>
```

Second, replace the whole `<ul> ... </ul>` list with:

FILE **pages/templates/pages/home.html (replace the <ul> list)**

```
    <ul>
        {% for skill in skills %}
        <li>{{ skill }}</li>
        {% endfor %}
    </ul>
```

WHY WE DO THIS The dictionary named `context` is a bag of data. The word on the left (for example `'skills'`) is the name you use inside the template. `{{ student_name }}` displays a value, and `{% for %}` repeats the HTML once for every item in the list.

CHECKPOINT The page looks the same as before, but now the name and skills come from Python. Test it: add `'CSS'` to the skills list in `views.py`, save, and refresh. The new skill appears without changing any HTML.

Summary: How It All Connects

File	Its job	Created in
`config/settings.py`	Lists the installed apps	Step 7
`config/urls.py`	Main address book. Passes requests to the app	Step 10
`pages/urls.py`	Connects each address to a view	Steps 9, 16
`pages/views.py`	Python functions that prepare each page	Steps 8, 13, 15, 20
`pages/templates/pages/*.html`	The HTML of each page	Steps 12, 14, 17, 19, 21
`pages/static/pages/style.css`	The colours and layout	Step 18

Journey of one request: Browser → `config/urls.py` → `pages/urls.py` → `views.py` → template (HTML + CSS) → Browser.

Practice Challenges

Try these on your own. Each one repeats the pattern you learned.

1. **Add a Contact page.** Create `contact.html`, add a `contact` view, add a URL `contact/`, and add a Contact link to the navigation bar of all three pages.
2. **Change the design.** Edit `style.css` to use your favourite colours. Save and refresh to see the result.
3. **Add hobbies.** In the `about` view, send a `hobbies` list through `context` and display it in `about.html` with a `{% for %}` loop.
4. **Add a new section.** Add a “My Projects” heading and list to the Home page.

Common Errors and How to Fix Them

Read the **last line** of the error message in the terminal or browser, then find it below.

What you see	Likely cause	Fix
`command not found` or `(venv)` is missing	Virtual environment is not active	Run the Activate command from Step 2
`No module named 'django'`	Django not installed, or venv not active	Activate venv, then `pip install django`
`does not appear to have any patterns`	`urlpatterns` is missing, misspelled, or the file is not saved	Check the spelling and save the file
`ImportError: cannot import name`	A file is empty or unsaved, or a name is misspelled	Save all files and check the spelling

What you see	Likely cause	Fix
`TemplateDoesNotExist`	Wrong template folder or file name	Path must be `pages/templates/pages/home.html`
`NoReverseMatch`	A URL `name=` does not match the name used in `{% url %}`	Compare both names and correct them
Page Not Found (404)	Wrong address typed	Use `/` for Home and `/about/` for About
CSS not applied	`{% load static %}` missing, or old page cached	Add it, restart the server, hard refresh
`Address already in use`	An old server is still running	Press Ctrl + C in the old terminal, or use `runserver 8001`
`TemplateSyntaxError`	A `{% %}` or `{{ }}` tag is not typed correctly	Check braces, percent signs, and `{% endfor %}`

Quick Command Reference

Task	Command
Activate the environment (Mac)	`source venv/bin/activate`
Activate the environment (Windows)	`venv\Scripts\activate`
Start the server	`python manage.py runserver`
Stop the server	Press Ctrl + C in the terminal
Create a new project	`django-admin startproject config .`
Create a new app	`python manage.py startapp appname`